

SwiGLU/GeGLU

导论

激活函数的发展，本质上是在不断解决神经网络的训练问题。最早的阶跃函数模拟神经元“是否激活”，但不可导，难以用于梯度训练；随后 **Sigmoid** 和 **Tanh** 引入连续可导形式，却带来了梯度消失问题。

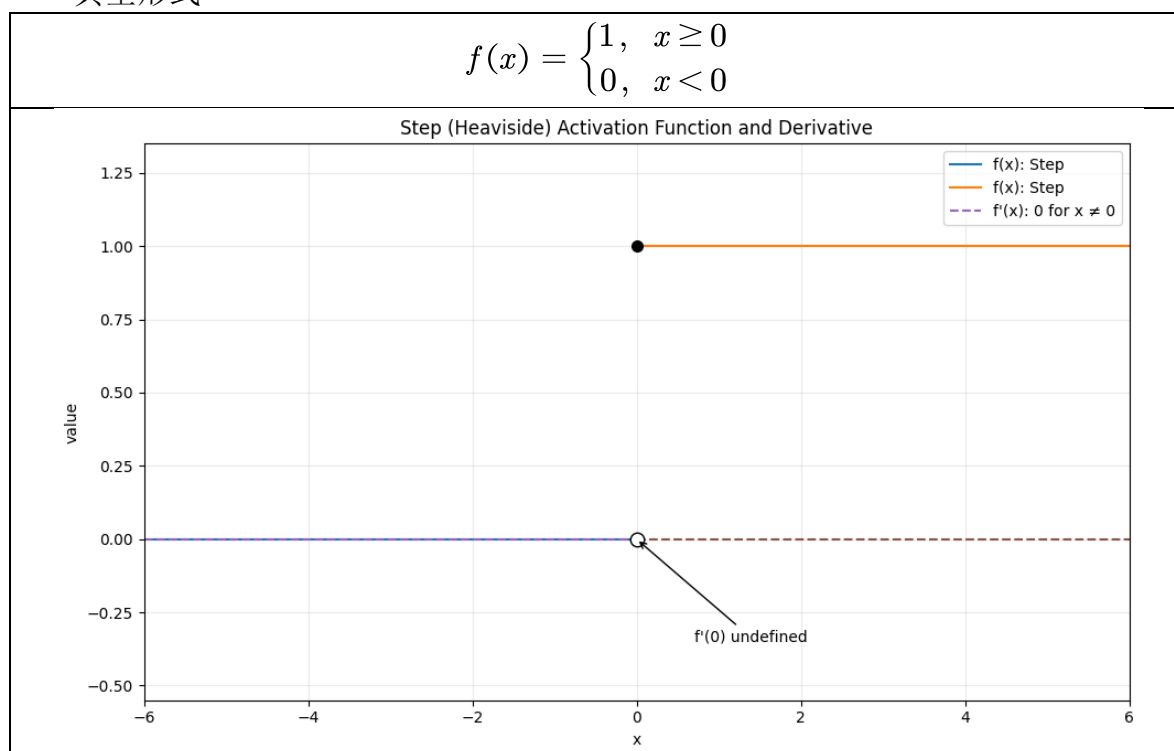
ReLU 的出现显著改善了深层网络的梯度传播，也推动了深度学习的快速发展。之后的 **Leaky ReLU**、**ELU**、**SELU** 等方法主要围绕负区间信息丢失和训练稳定性进行改进。

进入 **Transformer** 和大模型时代后，**GELU**、**SiLU/Swish** 更强调平滑性和自门控特性，而 **GeGLU**、**SwiGLU** 则进一步把激活函数发展成门控前馈结构的重要组成部分。

一、Threshold / Step Function

时间可以一直追溯到 **McCulloch 与 Pitts 1943 年的人工神经元模型**。他们使用的是“全或无”的神经元思想：输入超过阈值则激活，否则不激活。McCulloch–Pitts 的论文是人工神经网络历史上的奠基工作之一。

典型形式

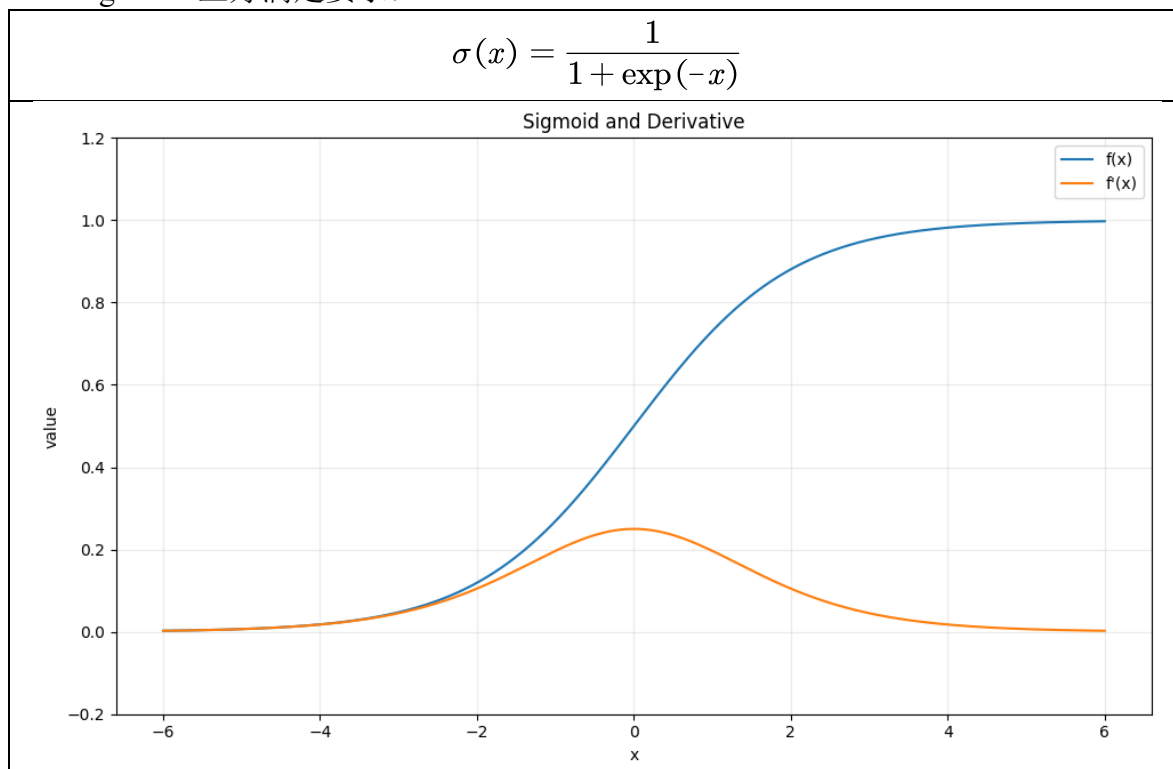


后来的 **Rosenblatt Perceptron** 仍然延续了这种阈值决策思想。Rosenblatt 在 1958 年发表了经典的 **Perceptron** 工作。

它最大的优点就是非常像早期人们想象中的生物神经元：要么 **firing**，要么不 **firing**。但是它有一个致命问题。几乎处处 $f'(x) = 0$ ，而在 $x=0$ 处甚至不可导。处甚至不可导。

二、Sigmoid

Sigmoid 正好满足要求:



导数非常漂亮 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, Rumelhart、Hinton、Williams 1986 年的经典 Nature 工作系统展示了通过反向传播训练多层神经网络的机制, 这是 Sigmoid 类可微神经元大规模流行的重要历史背景。

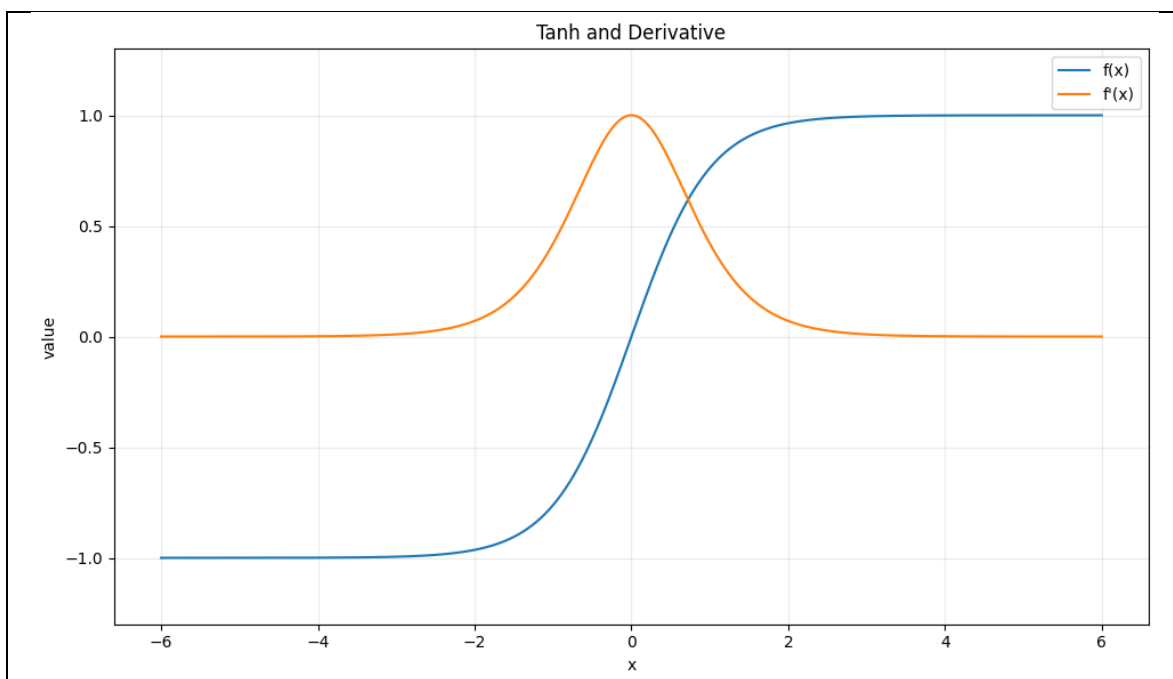
Sigmoid 的输出 $0 < \sigma(x) < 1$, 所以特别适合解释成激活概率或者门开启的程度, 这也是为什么今天 Sigmoid 虽然退出了大多数隐藏层, 却仍然大量存在于 LSTM Gate; GRU Gate; Attention Gate; GLU Gate; Binary Classification Output

Sigmoid 最大的问题: 梯度消失

当 $x \rightarrow +\infty$ 或者 $x \rightarrow -\infty$, 都有 $\sigma'(x) \rightarrow 0$, 于是深层网络梯度不断连乘, 最终 $\frac{\partial L}{\partial W_1} \approx 0$, 另外 Sigmoid 不是零中心 $\sigma(x) > 0$, 这也会使优化方向存在偏置。

三、Tanh

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

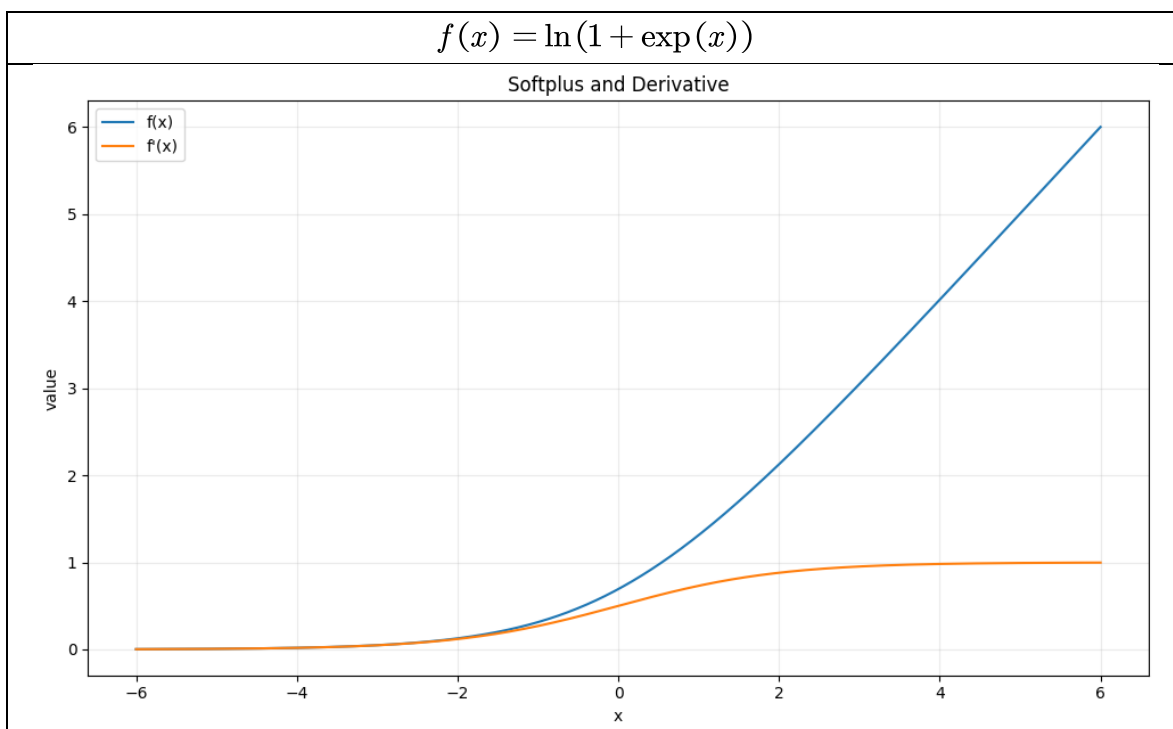


范围 $-1 < \tanh(x) < 1$ ，导数 $\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$ ，相比 Sigmoid，它最大的改进是 $\text{mean} \approx 0$ ，LeCun 等人的经典 *Efficient BackProp* 对零中心输入和对称激活进行了系统讨论，并长期影响了神经网络训练实践。

但是 $x \rightarrow \pm \infty$ ，仍然导致 $\tanh'(x) \rightarrow 0$ ，所以饱和问题依旧存在。

Sigmoid 和 tanh 最明显的共同特征就是两端都逐渐变平，这就是“饱和”。

四、Softplus



它的重要性质是 $f'(x) = \sigma(x)$ ，Softplus 在 Dugas 等人的工作中被使用并系统化

为平滑的正值非线性。它实际上可以理解成平滑版 ReLU。当 $x \gg 0$ ，有 $\ln(1 + \exp(x)) \approx x$ ，当 $x \ll 0$ ，有 $\ln(1 + \exp(x)) \approx 0$ ，所以 Softplus 非常接近后来大获成功的 ReLU

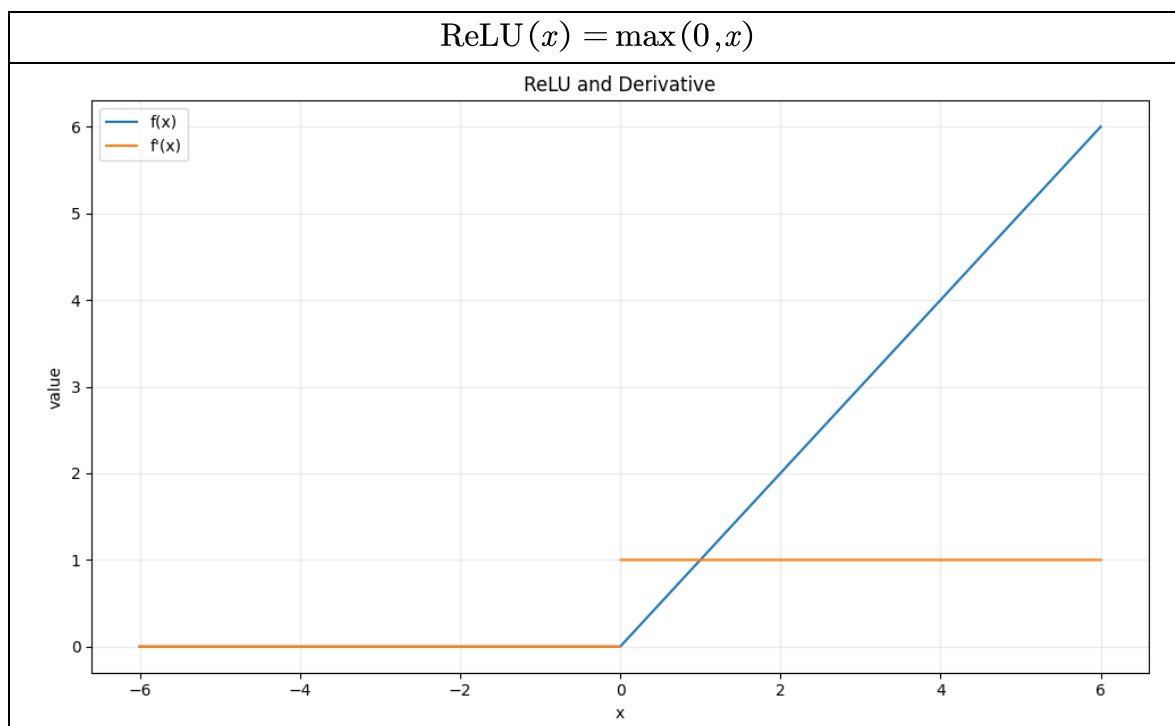
优点

平滑、处处可导，没有 ReLU 在零点的尖角。

缺点

计算中存在 $\exp(x)$ 和 $\ln(x)$ ，早期硬件上明显比简单的 ReLU 贵。

五、ReLU



导数 $\text{ReLU}'(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases}$ ，Nair 与 Hinton 2010 年在 ICML 的工作证明了

Rectified Linear Units 在 Restricted Boltzmann Machines 中的有效性；随后 Glorot、Bordes、Bengio 2011 年的 Deep Sparse Rectifier Neural Networks 进一步推动了 rectifier 网络。

ReLU 有三个巨大优势。第一，当 $x > 0$ 时： $f'(x) = 1$ ，不再像 Sigmoid： $f'(x) \rightarrow 0$ ，因此梯度传播容易得多。第二，计算极便宜， $\max(0, x)$ 。第三，它天然产生 $f(x) = 0$ 的稀疏激活。这使深层 CNN 的训练变得非常有效。

ReLU 最大的问题：Dying ReLU

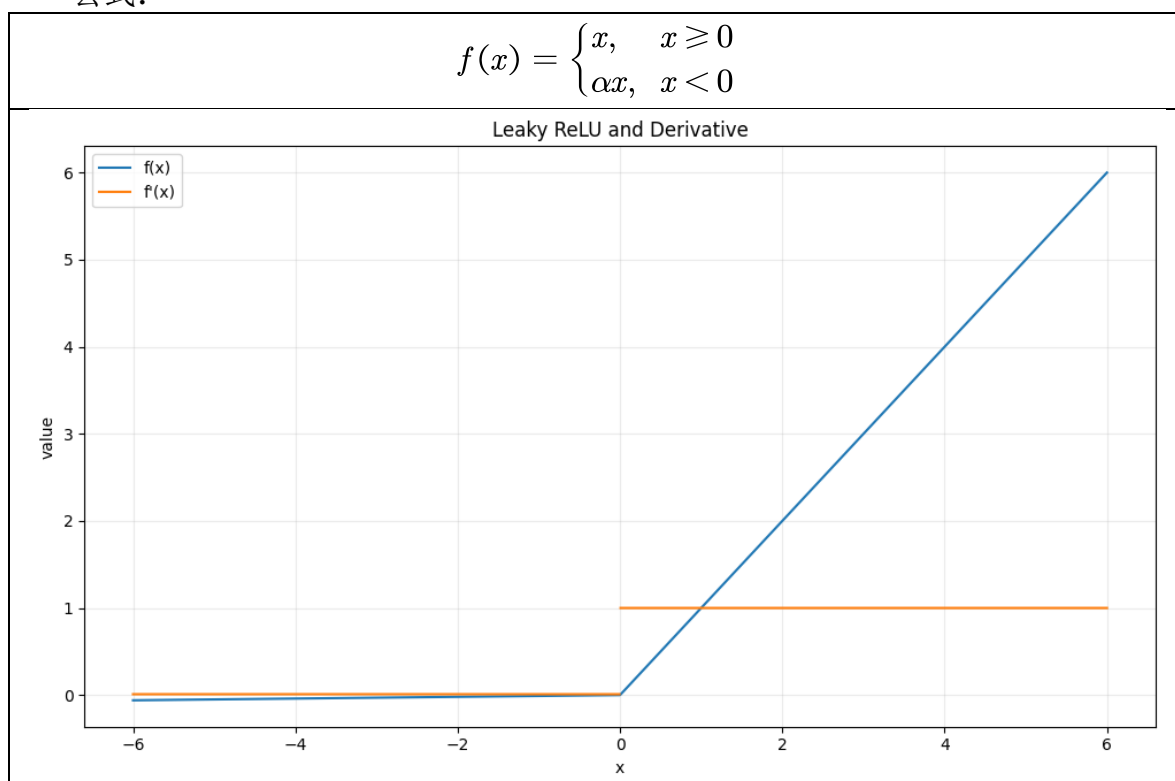
如果一个神经元长期落到 $x < 0$ ，那么 $f(x) = 0$ ，并且 $f'(x) = 0$ ，于是参数无法更新，这就是 Dying ReLU。于是一大批 ReLU 改进型开始出现。

六、Leaky ReLU

负数不要直接砍成 0，给它留一点梯度。

公式：

$$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$$



例如 $\alpha = 0.01$ 或者 $\alpha = 0.1$ ，Maas、Hannun、Ng 等人在 2013 年的语音识别工作中系统研究了这种 rectifier nonlinearity。

负区域 $f'(x) = \alpha$ 不再等于零。

所以优点是基本解决 Dying ReLU；缺点是 α 是人手指定的。那么自然就有人问：为什么不让网络自己学？

七、PReLU

于是 He Kaiming 等人在 2015 年提出 PReLU：

$$f(x) = \begin{cases} x, & x \geq 0 \\ ax, & x < 0 \end{cases}$$

a 是可学习参数。

论文：*Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification* 发表于 ICCV 2015。

网络可以自己学习到底应该漏多少负值。

同时这篇工作还与今天非常重要的 He Initialization 密切相关。

八、RRReLU

训练时 $a \sim U(l, u)$ ，然后

$$f(x) = \begin{cases} x, & x \geq 0 \\ ax, & x < 0 \end{cases}$$

也就是负斜率随机。Xu 等人的 2015 年工作比较了 ReLU、Leaky ReLU、PReLU 与 RReLU，并提出随机负斜率可以起到一定的正则化作用。

思想类似 Dropout 通过随机性避免过拟合，但今天它远不如 ReLU/SiLU/GELU 常见。

九、Maxout

Goodfellow 等人在 2013 年提出 Maxout:

$$h_i(x) = \max_{j=1,\dots,k} z_{ij}$$

其中 $z_{ij} = x^T W_{ij} + b_{ij}$

论文 *Maxout Networks*

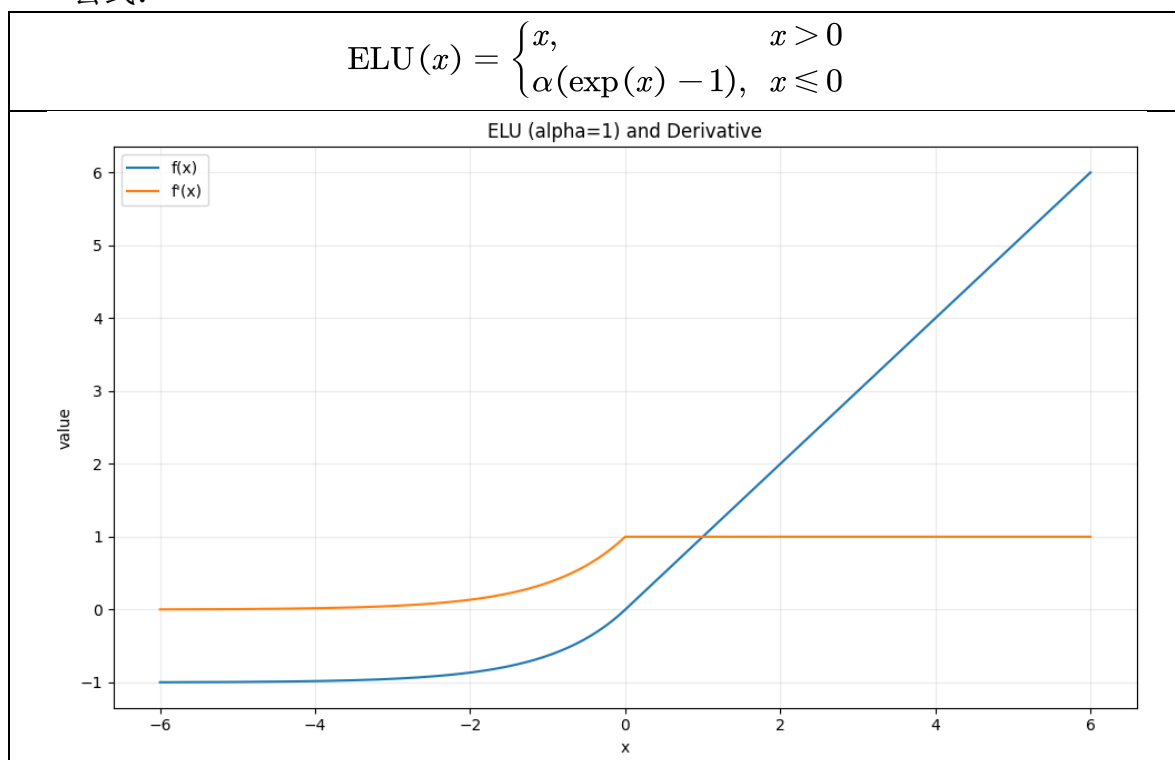
它的思想非常激进：不提前规定激活函数形状，让网络通过多个线性函数的最大值学习一个分段线性函数。理论表达能力很强。但问题是如果普通神经元需要一个 $Wx + b$ ，Maxout 需要 k 套，参数量和计算量明显增加，所以它并没有成为今天 LLM 的主流。

十、ELU

2015 年 Clevert、Unterthiner、Hochreiter 提出：Exponential Linear Unit。

公式：

$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \alpha(\exp(x) - 1), & x \leq 0 \end{cases}$$



和 Leaky ReLU 最大区别是负区域不是一条直线，而是一条逐渐饱和的指数曲线。当 $x \rightarrow -\infty$ ，有 $\text{ELU}(x) \rightarrow -\alpha$

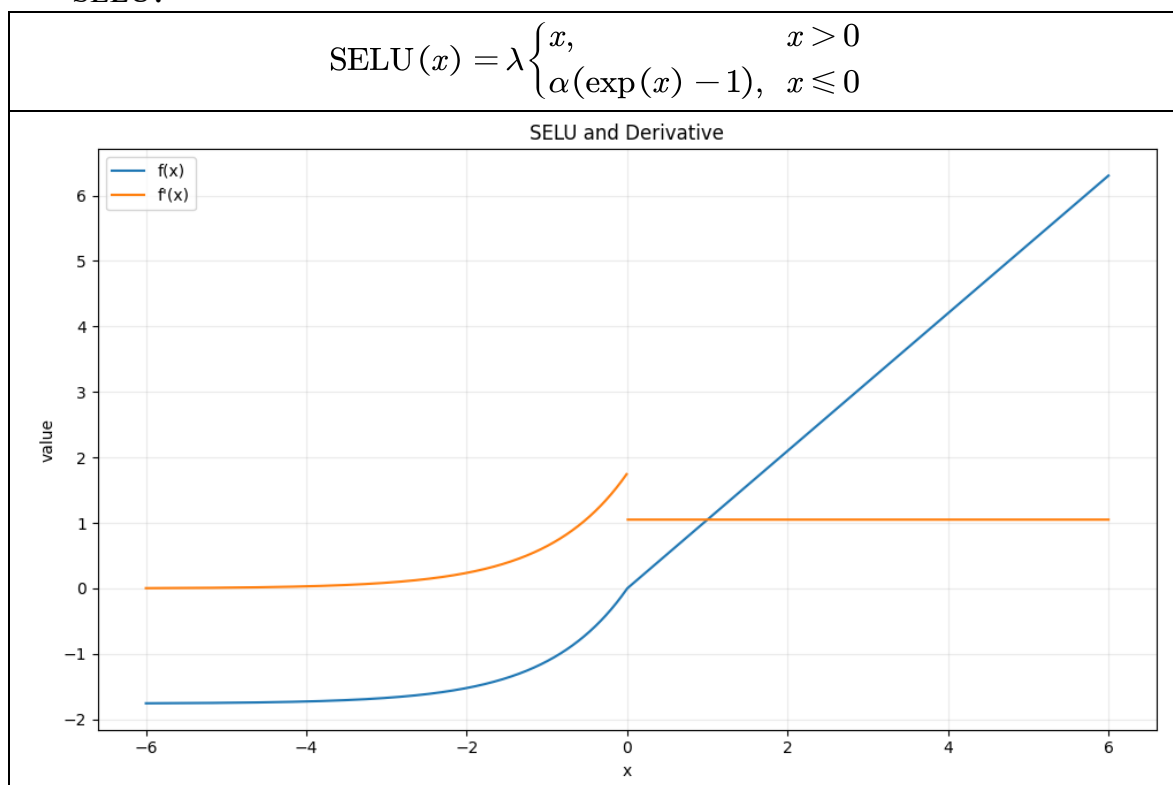
优点是负激活可以帮助输出均值更接近零，原论文也把这一点作为其优化优势的

重要解释：缺点是需要 $\exp(x)$ ，计算成本比 ReLU 高。

十一、SELU

2017 年 Klambauer 等人提出： Self-Normalizing Neural Networks。

SELU:



其中典型参数大约 $\lambda \approx 1.0507$ ， $\alpha \approx 1.6733$

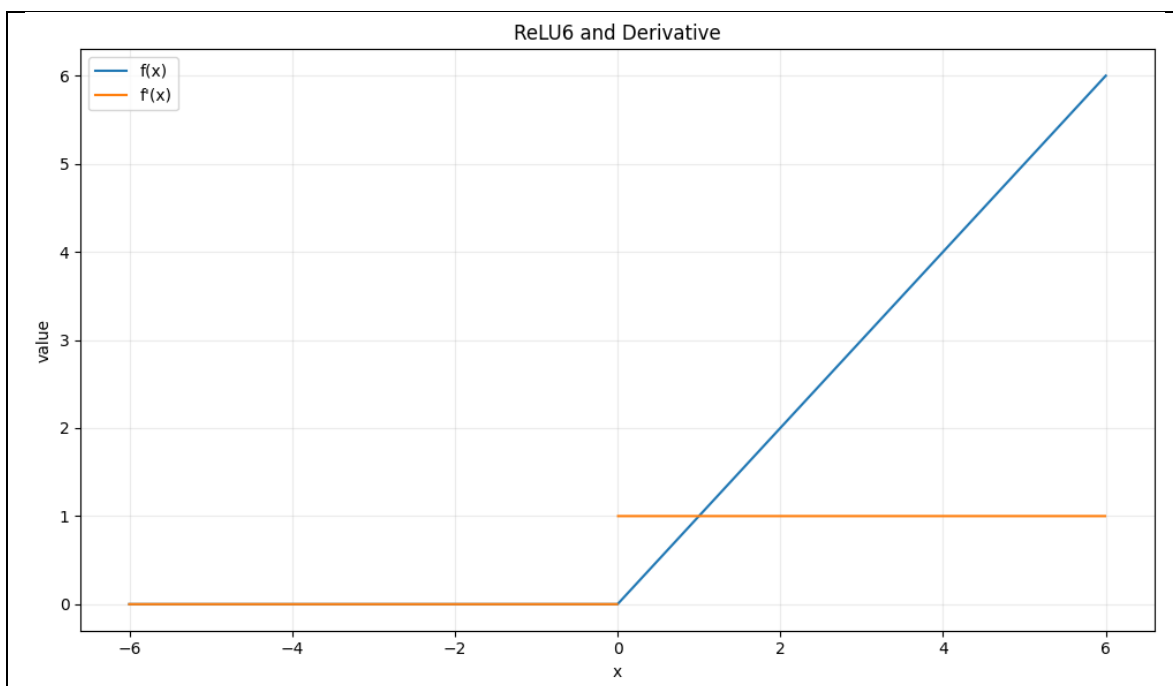
其中输入均值为 0，方差为 1 的样本最后得到的输出也为均值为 0，方差为 1 的样本，也就是：Self-Normalizing。

论文从固定点与收缩映射角度论证了这种性质，但是它依赖一系列架构和初始化条件，因此没有取代 LayerNorm / RMSNorm + 现代激活函数的路线。

十二、ReLU6

ReLU6:

$$\text{ReLU6}(x) = \min(\max(0, x), 6)$$



MobileNet 系列大量采用了这种受限 ReLU, 尤其适合低精度与移动端网络设计。

优点:

范围有限, 硬件实现简单, 对低精度计算友好。

缺点:

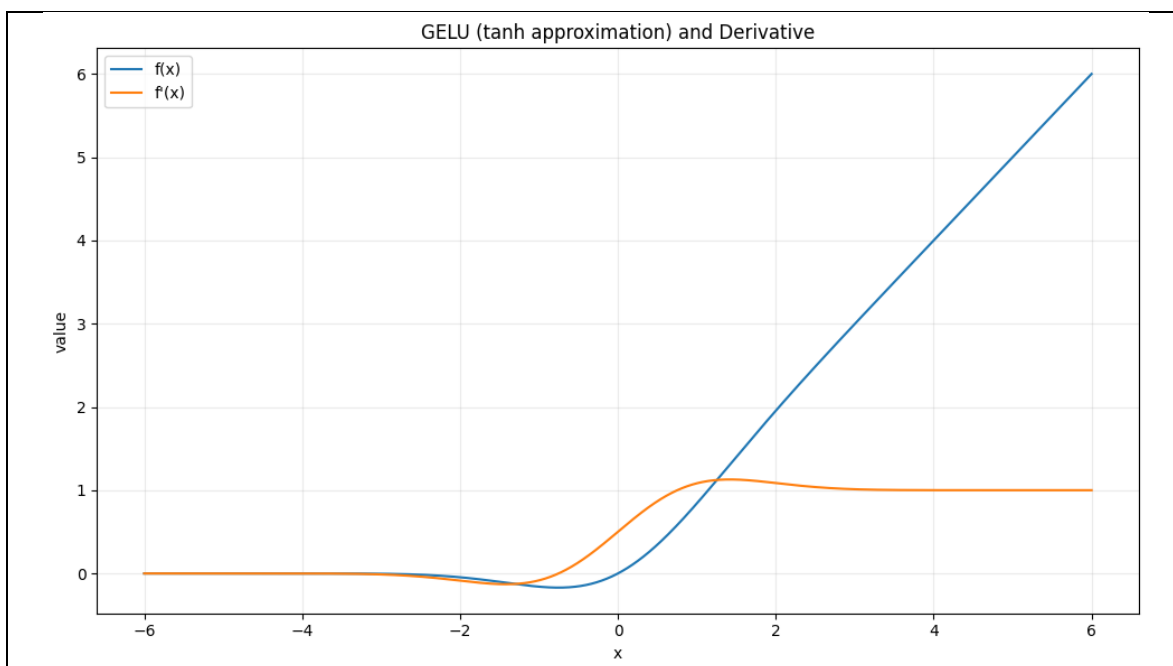
当 $x > 6$ 时, 又出现 $f'(x) = 0$, 所以正半轴也产生饱和区域。

十三、GELU

Hendrycks 与 Gimpel 在 2016 年提出: Gaussian Error Linear Unit。

公式:

$$\text{GELU}(x) = x \Phi(x)$$



其中 $\Phi(x)$ 是标准正态分布 CDF。也可以写成: $\Phi(x) = \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$, 所以

$$\operatorname{GELU}(x) = \frac{x}{2} \left[1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

$$\text{常用 tanh 近似 } \operatorname{GELU}(x) \approx \frac{x}{2} \left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right) \right]$$

$\operatorname{GELU} x \Phi(x)$ 相当于根据输入大小平滑地决定: 这个信息应该保留多少。所以它是一种 Smooth Gating。GELU 原论文在视觉、语言和语音任务上对 ReLU/ELU 进行了比较, 并报告了良好的结果。

优点

平滑; 允许小量负值; 不会突然在 $x=0$ 发生硬截断; 优化行为通常比 ReLU 更柔和。

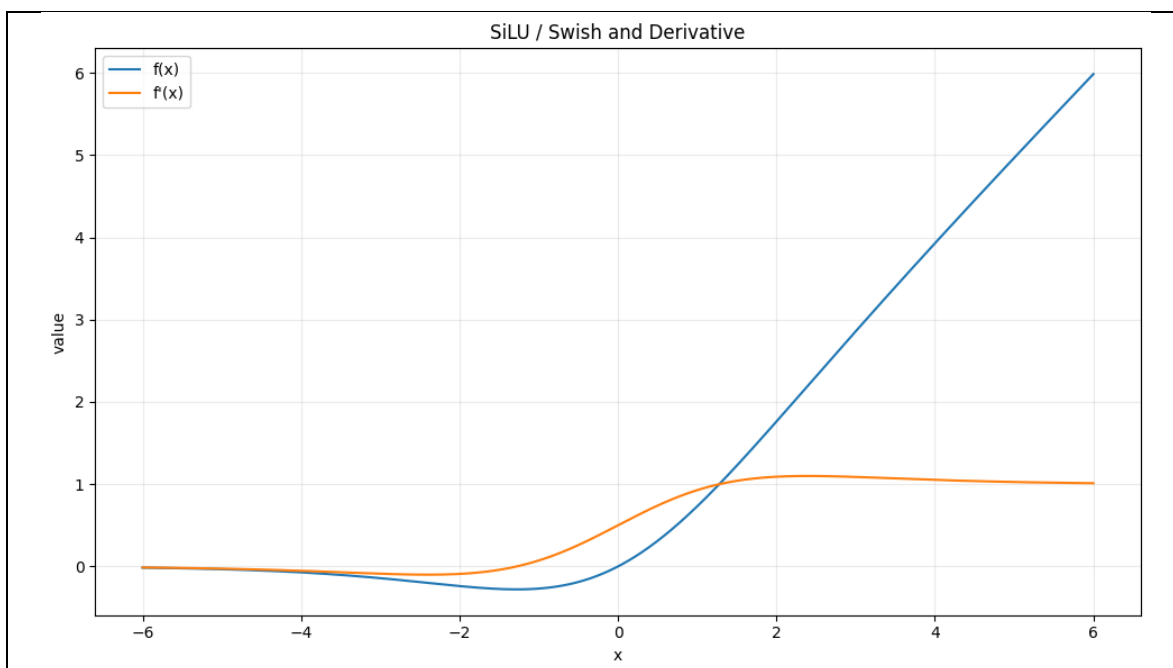
缺点

比 $\max(0, x)$ 计算复杂。

十四、 SiLU

2017 年 Elfwing、Uchibe、Doya 提出 Sigmoid-Weighted Linear Unit:

$$\operatorname{SiLU}(x) = x \sigma(x)$$



也就是 $\text{SiLU}(x) = \frac{x}{1 + \exp(-x)}$

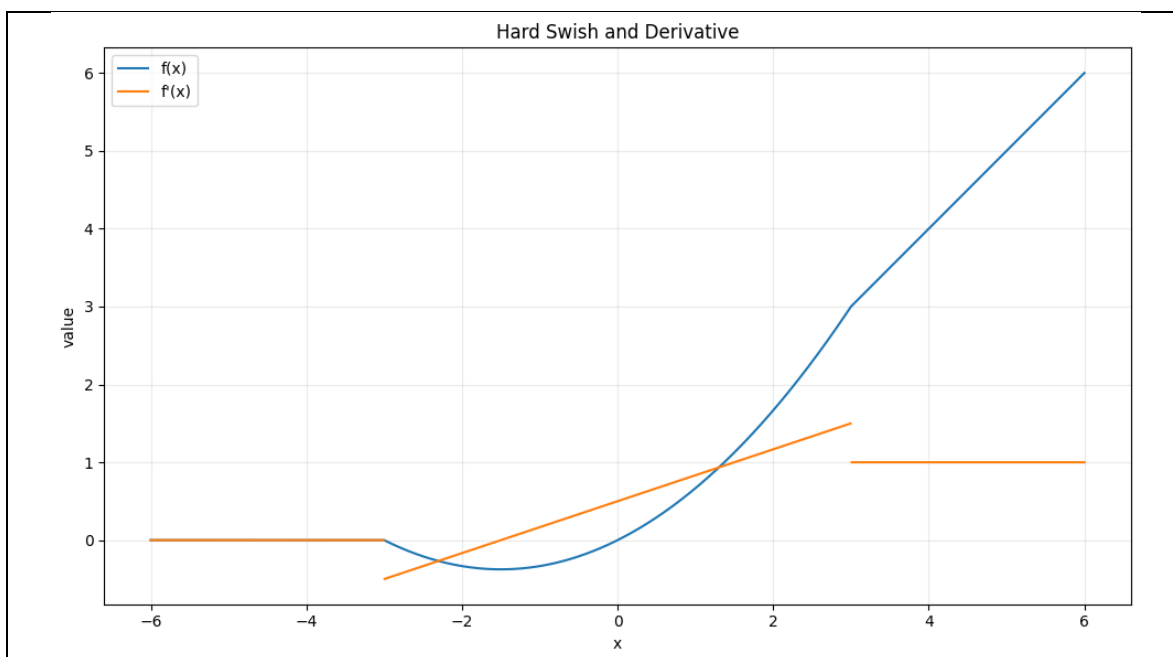
几乎同一时期，Ramachandran、Zoph、Le 使用自动搜索发现了 $f(x) = x\sigma(\beta x)$ ，并将其称为 Swish。当 $\beta = 1$ 时： $\text{Swish}(x) = \text{SiLU}(x)$ ，所以今天工程语境里经常近似把 SiLU 和 Swish 视作同一个东西。严格来说，Swish 是 $x\sigma(\beta x)$ 这个函数族。

考虑 $f(x) = x\sigma(x)$ ，它可以理解成输入自己给自己做 Gate，所以也叫 Self-Gated Activation，它还有一个重要特征非单调，在稍微小于 0 的区域 $f(x) < 0$ ，但不会像 ReLU 一样直接全部归零，这条“小负尾巴”后来成为很多现代激活函数的重要共同特征。

十五、Hard-Swish

Swish 的缺点是 $\sigma(x)$ 涉及指数运算，移动端嫌贵。于是 MobileNetV3 使用：

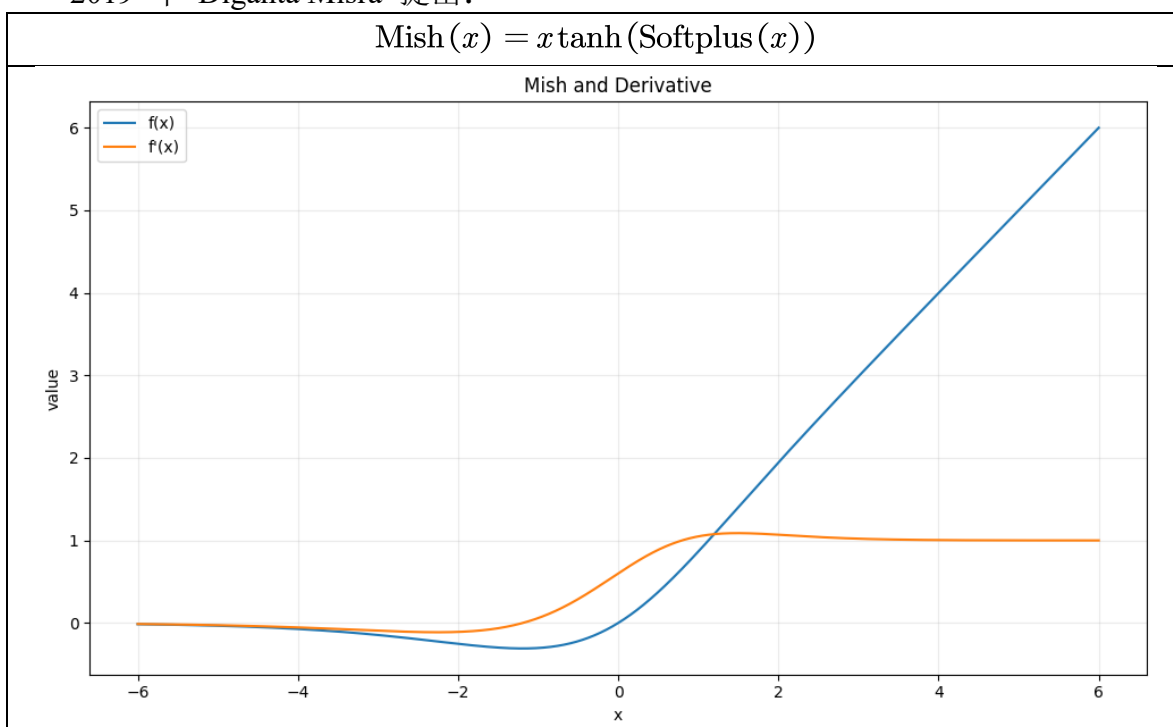
$$\text{h-swish}(x) = x \frac{\text{ReLU6}(x + 3)}{6}$$



来近似 Swish。MobileNetV3 论文明确讨论了这一硬件友好的 hard-swish 设计。所以它本质上是 Mobile; Edge AI; Embedded 但并不是大型 Transformer 的主流选择。

十六、Mish

2019 年 Diganta Misra 提出：



也就是 $\text{Mish}(x) = x \tanh(\ln(1 + \exp(x)))$ ，和 SiLU 一样：平滑；非单调；允许负值；正半轴近似线性，从纯曲线性质来看非常漂亮。但是计算需要： $\exp$ ； $\log$ ； $\tanh$ ，相对昂贵。

所以虽然它在部分 CV 模型中取得过不错表现，但没有成为大模型 FFN 的统治性选择。原论文报告了多个视觉任务上的实验收益。

十七、GLU

到这里为止，我们讲的基本都是： $y = f(x)$ ，一个输入，一个输出。但是现代 Transformer 开始出现另一种思路：不再单纯问“应该用什么 $f(x)$ ”，而是增加一整条 Gate 分支。

Dauphin 等人在 2016/2017 年提出 Gated Linear Unit，并用于语言模型。GLU：

$$\text{GLU}(x) = (xW + b) \odot \sigma(xV + c)$$

注意：这已经不是普通的 $f(x)$ 了，而是一条支路负责内容，一条支路负责门，然后 $\odot$ 逐元素相乘。GLU 的语言建模工作报告了更快收敛和更低 perplexity 的结果。

十八、ReGLU

$$\text{ReGLU}(x) = \text{ReLU}(xW) \odot xV$$

十九、GEGLU

把 ReLU 换成 GELU：

$$\text{GEGLU}(x) = \text{GELU}(xW) \odot xV$$

二十、SwiGLU

再把 GELU 换成 Swish/SiLU：

$$\text{SwiGLU}(x) = \text{SiLU}(xW) \odot xV$$

如果加上输出投影，现代 Transformer FFN 实际更接近：

$$\text{FFN}(x) = W_2[\text{SiLU}(W_g x) \odot (W_u x)]$$

Shazeer 的实验中，GEGLU 和 SwiGLU 是表现最好的 GLU 变体之一。

截至 2026 年，SiLU 已经成为先进开放大语言模型中最主流的 Gate 激活之一，而真正的 FFN 结构通常进一步演化成 SwiGLU-style gated MLP。